

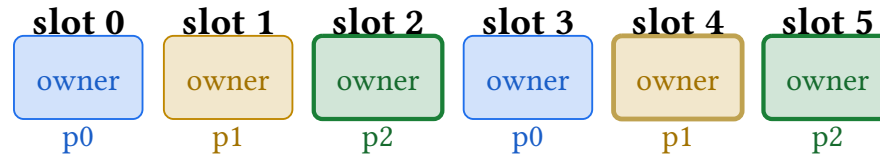
An Agreement bug in Mencius (Optimizations 1 + 3)

How a false suspicion makes correct servers commit different values for one slot

Mencius (Mao, Junqueira, Marzullo, OSDI 2008) – full algorithm, Appendix C

Refresher 1/3 – the replicated log

Mencius replicates a **log** of slots, each decided by one Paxos instance. Slots are pre-assigned round-robin to a **coordinator**, $\text{owner}(i) = i \bmod n$ (here $n = 3$):

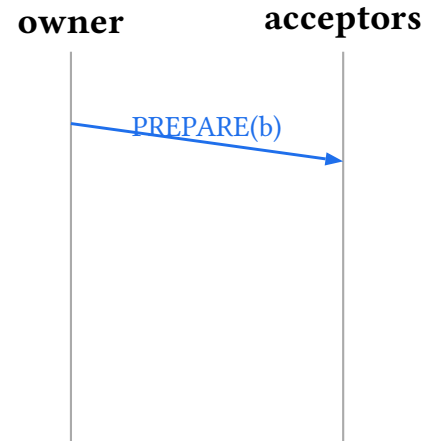


- **p2** owns slots **2, 5, 8, ...** – p1 will revoke this whole **block**.
- **p1** owns slot **4** – the one door the bug walks through.
- Commit is strictly in log order; the violation will be on **slot 2**.

Refresher 2/3 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b, ab, av)	Phase 1b: promise + last accept
PROPOSE(b, v)	Phase 2a: accept v at b
ACCEPT(b, v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen

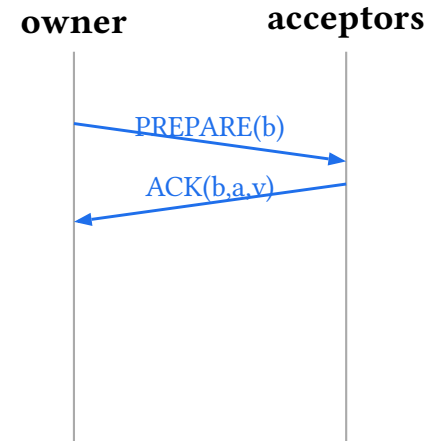


SUGGEST = PROPOSE(0, v) (real value); **SKIP** = PROPOSE(0, no-op) (empty). The owner uses ballot 0 with no Phase 1.

Refresher 2/3 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b, a, v)	Phase 1b: promise + last accept
PROPOSE(b, v)	Phase 2a: accept v at b
ACCEPT(b, v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen

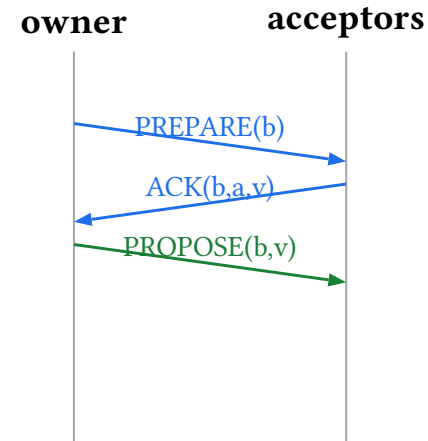


SUGGEST = PROPOSE(0, v) (real value); **SKIP** = PROPOSE(0, no-op) (empty). The owner uses ballot 0 with no Phase 1.

Refresher 2/3 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b,a,v)	Phase 1b: promise + last accept
PROPOSE(b,v)	Phase 2a: accept v at b
ACCEPT(b,v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen

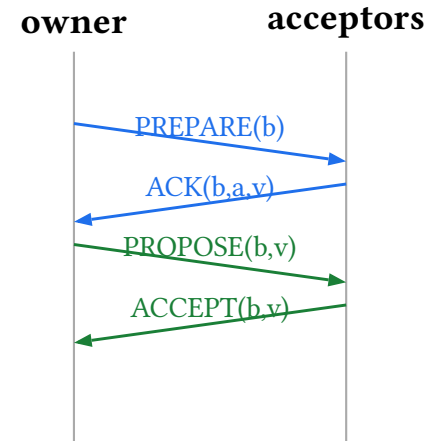


SUGGEST = `PROPOSE(0, v)` (real value); **SKIP** = `PROPOSE(0, no-op)` (empty). The owner uses ballot 0 with no Phase 1.

Refresher 2/3 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b,a,v)	Phase 1b: promise + last accept
PROPOSE(b,v)	Phase 2a: accept v at b
ACCEPT(b,v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen

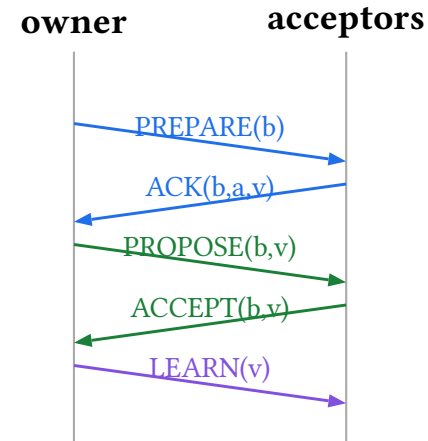


SUGGEST = PROPOSE(0, v) (real value); **SKIP** = PROPOSE(0, no-op) (empty). The owner uses ballot 0 with no Phase 1.

Refresher 2/3 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b,a,v)	Phase 1b: promise + last accept
PROPOSE(b,v)	Phase 2a: accept v at b
ACCEPT(b,v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen



SUGGEST = **PROPOSE**(0, v) (real value); **SKIP** = **PROPOSE**(0, no-op) (empty). The owner uses ballot 0 with no Phase 1.

Refresher 3/3 – accept test, quorum, and revoke

Accept. A server accepts $\text{PROPOSE}(b, v)$ **only if**

$\text{prepared_ballot} \leq b$ and $\text{accepted_ballot} < b$

Refresher 3/3 – accept test, quorum, and revoke

Accept. A server accepts $\text{PROPOSE}(b, v)$ **only if**

$$\text{prepared_ballot} \leq b \text{ and } \text{accepted_ballot} < b$$

Chosen. A value is locked forever once a **quorum** (2 of 3) accepts it at one ballot. The quorum holder broadcasts $\text{LEARN}(v)$.

Refresher 3/3 – accept test, quorum, and revoke

Accept. A server accepts $\text{PROPOSE}(b, v)$ **only if**

$$\text{prepared_ballot} \leq b \text{ and } \text{accepted_ballot} < b$$

Chosen. A value is locked forever once a **quorum** (2 of 3) accepts it at one ballot. The quorum holder broadcasts $\text{LEARN}(v)$.

Revoke. A server that **suspects** the coordinator grabs a **higher** ballot and runs $\text{PREPARE}(b)$, bumping recipients' prepared_ballot to b .

Refresher 3/3 – accept test, quorum, and revoke

Accept. A server accepts $\text{PROPOSE}(b, v)$ **only if**

$$\text{prepared_ballot} \leq b \text{ and } \text{accepted_ballot} < b$$

Chosen. A value is locked forever once a **quorum** (2 of 3) accepts it at one ballot. The quorum holder broadcasts $\text{LEARN}(v)$.

Revoke. A server that **suspects** the coordinator grabs a **higher** ballot and runs $\text{PREPARE}(b)$, bumping recipients' prepared_ballot to b .

This is the problem. After a revoke raises prepared_ballot to 1, the owner's later ballot-0 SUGGEST fails the test ($1 \leq 0$ is false) and is **silently rejected** – Coordinated Paxos has no NACK.

Optimization 1 – infer skips instead of sending them

Broadcasting an explicit SKIP for every empty slot is wasteful. So each server keeps `est_index[q]` = “the lowest q-owned slot I have not yet accounted for”, and **infers** skips:

Optimization 1 – infer skips instead of sending them

Broadcasting an explicit SKIP for every empty slot is wasteful. So each server keeps `est_index[q]` = “the lowest q-owned slot I have not yet accounted for”, and **infers** skips:

When I process a message from q about its slot i, every q-owned slot below i that I have not learned, I conclude q **skipped** – so I write no-op into those slots myself, and advance `est_index[q]`.

Optimization 1 – infer skips instead of sending them

Broadcasting an explicit SKIP for every empty slot is wasteful. So each server keeps `est_index[q]` = “the lowest q-owned slot I have not yet accounted for”, and **infers** skips:

When I process a message from q about its slot i, every q-owned slot below i that I have not learned, I conclude q **skipped** – so I write no-op into those slots myself, and advance `est_index[q]`.

Justification (FIFO, Lemmas 5/6). By the time q’s message about slot i reaches me, I have already received everything q sent earlier. If q had suggested a real value to an earlier slot j, I would know it. I do not – therefore q skipped j.

The hidden assumption – and how revoke breaks it

The inference equates “**I never processed a real suggestion from q for slot j**” with “**q never sent one.**”

The hidden assumption – and how revoke breaks it

The inference equates “**I never processed a real suggestion from q for slot j**” with “**q never sent one.**”

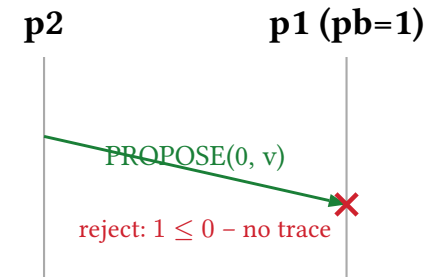
That holds in the base protocol. **Revocation breaks it:** a revoke can make me **receive** q’s real suggestion and then **throw it away** on the ballot test – recording **nothing**.

The hidden assumption – and how revoke breaks it

The inference equates “**I never processed a real suggestion from q for slot j**” with “**q never sent one.**”

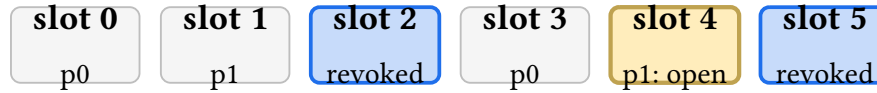
That holds in the base protocol. **Revocation breaks it:** a revoke can make me **receive** q’s real suggestion and then **throw it away** on the ballot test – recording **nothing**.

The suggestion was sent and received, yet left **no trace**. The inference then fabricates a no-op over a slot the coordinator genuinely **used**.



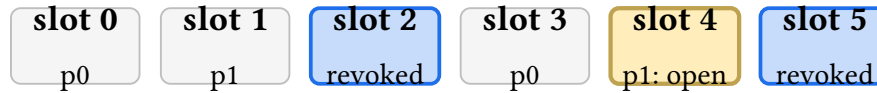
Optimization 3 – why slot 4 is the only door

`OnSuspect(q)` does not revoke one slot. It revokes a whole **β -window** block of `q`-owned slots (paper uses $\beta = 100000$; we use 2, window `[2, 5]`):



Optimization 3 – why slot 4 is the only door

`OnSuspect(q)` does not revoke one slot. It revokes a whole **β -window** block of `q`-owned slots (paper uses $\beta = 100000$; we use 2, window `[2, 5]`):



- Suspecting `p2` revokes **2 and 5** – a later `SUGGEST` to slot 5 is rejected too, so the bug **cannot** route through slot 5.
- `p1` never revokes its **own** slots. **Slot 4 stays at ballot 0** – the path the trace must take.

The setup

- Three servers **p0**, **p1**, **p2**; quorum = 2; FIFO links, **no crashes**.
- p1 **falsely suspects** p2 (the paper's detector is only **eventually** accurate). p2 is **alive and correct**.
- Slot 2 unlearned everywhere; on p1, `est_index[p2] = 2`, `my_index = 1`.
- Goal: a **divergence** on slot 2.

slot 2	pb	ab	av	learned
p0	0	-1	⊥	⊥
p1	0	-1	⊥	⊥
p2	0	-1	⊥	⊥

p1's `est_index[p2] = 2`

Phase A – p1 falsely suspects p2 and block-revokes {2, 5}

p0



p1



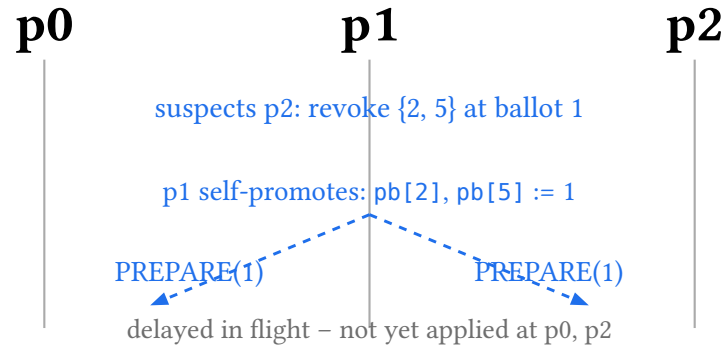
p2



suspects p2: revoke {2, 5} at ballot 1

p1 self-promotes: pb[2], pb[5] := 1

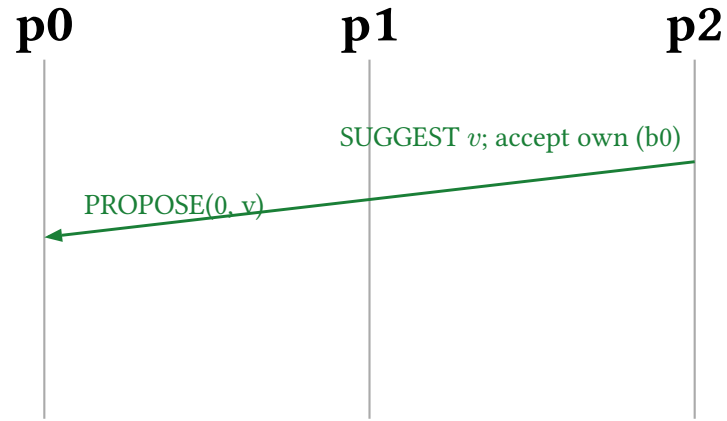
Phase A – p1 falsely suspects p2 and block-revokes {2, 5}



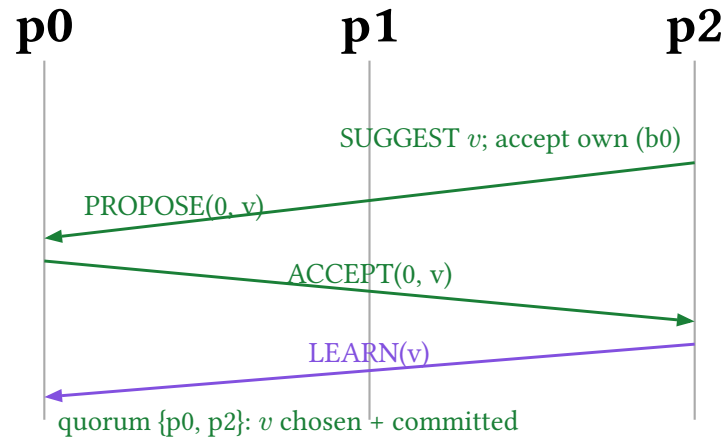
slot 2	pb	ab	av	learned
p0	0	−1	⊥	⊥
p1	1	−1	⊥	⊥
p2	0	−1	⊥	⊥

p1's est_index[p2] = 2

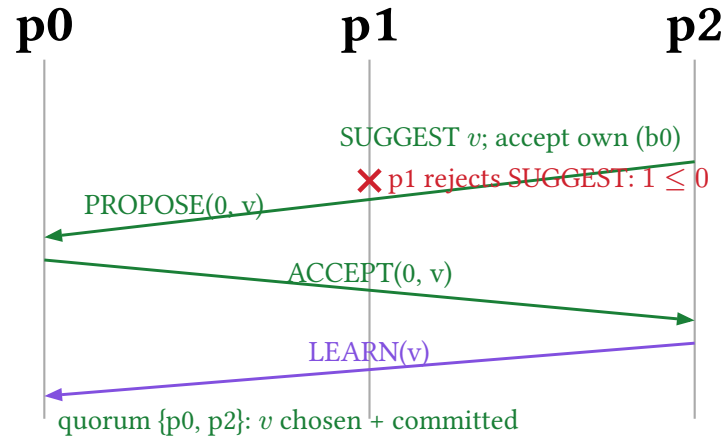
Phase B+C – {p0, p2} choose v at slot 2; p1 rejects



Phase B+C – {p0, p2} choose v at slot 2; p1 rejects



Phase B+C – {p0, p2} choose v at slot 2; p1 rejects

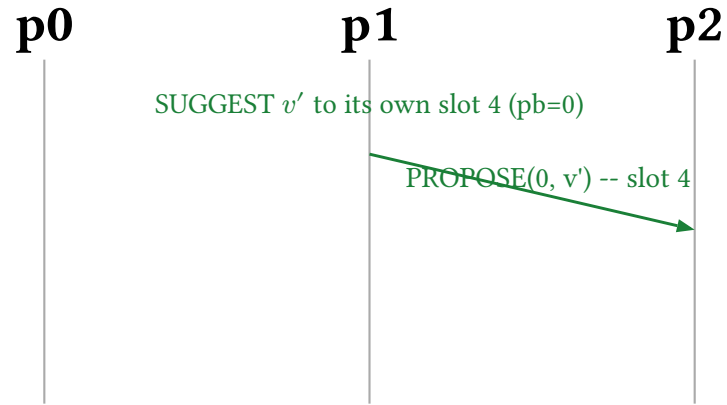


p1 records **nothing** – the committed v leaves no mark on it, so `est_index[p2]` stays 2 (stale).

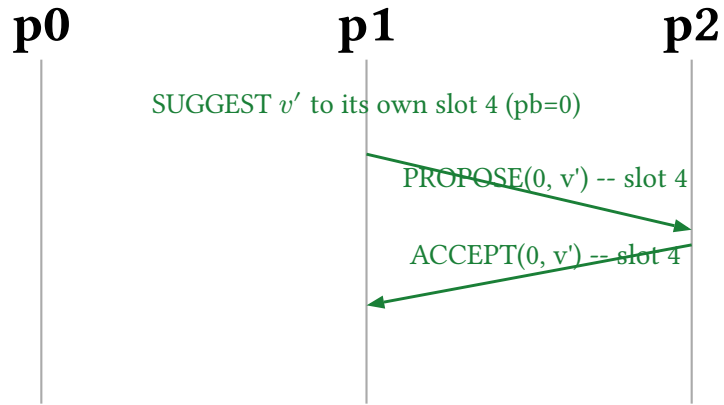
slot 2	pb	ab	av	learned
p0	0	0	v	v
p1	1	-1	⊥	⊥
p2	0	0	v	v

p1's `est_index[p2]` = 2

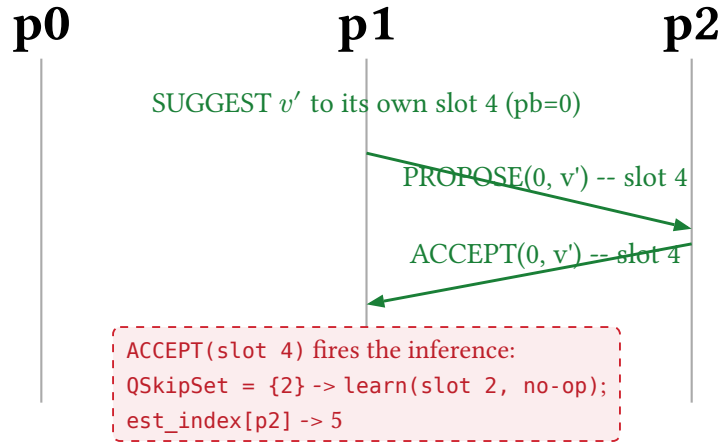
Phase D – slot 4 triggers a fabricated no-op for slot 2



Phase D – slot 4 triggers a fabricated no-op for slot 2



Phase D – slot 4 triggers a fabricated no-op for slot 2

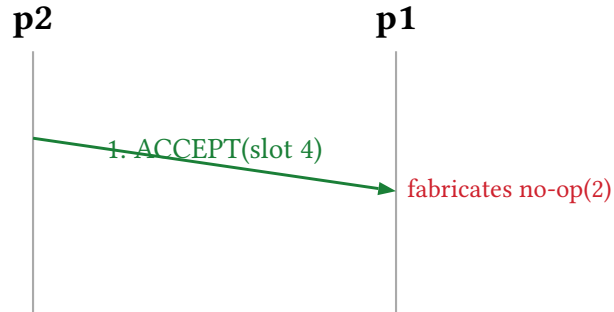


slot 2	pb	ab	av	learned
p0	0	0	v	v
p1	1	-1	$\perp$	no-op
p2	0	0	v	v

p1's `est_index[p2]` = 5

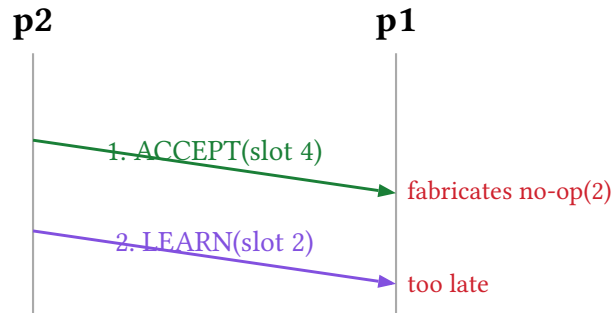
Why the LEARN is legitimately late (FIFO is not broken)

On the **p2 -> p1** channel, p2 sends ACCEPT(slot 4) the moment it gets p1's SUGGEST(4), but sends LEARN(slot 2) only **after** collecting p0's ACCEPT for slot 2. FIFO then delivers them **in that order**:



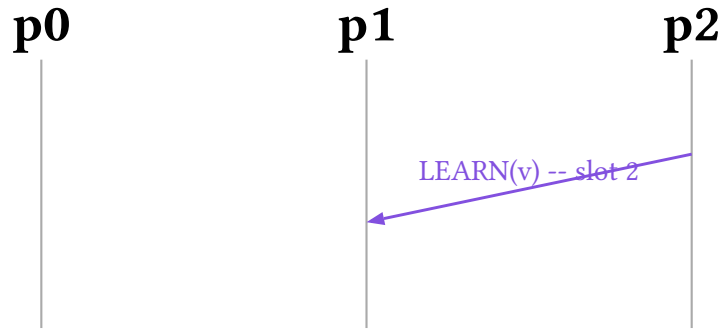
Why the LEARN is legitimately late (FIFO is not broken)

On the **p2 -> p1** channel, p2 sends ACCEPT(slot 4) the moment it gets p1's SUGGEST(4), but sends LEARN(slot 2) only **after** collecting p0's ACCEPT for slot 2. FIFO then delivers them **in that order**:

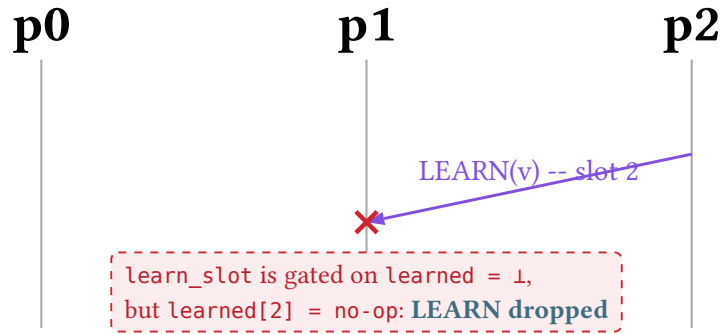


Arrange p1's SUGGEST(4) to reach p2 before p0's ACCEPT(2) does – then the channel order is exactly this. No FIFO rule is broken.

Phase E – the real outcome arrives too late



Phase E – the real outcome arrives too late



slot 2	pb	ab	av	learned
p0	0	0	v	v
p1	1	-1	$\perp$	no-op
p2	0	0	v	v

p1's est_index[p2] = 5

Two correct servers commit **v**.
A third correct server commits **no-op**.

Same slot, no crash, no message loss – only a false suspicion and a concurrent block-revoke. **Agreement** and **Total Order** are broken, permanently.

Why none of the safety nets catch it

Rule 4 (re-suggest on no-op) fires only at the **proposer** whose value was overwritten by a no-op. The proposer is p2, and p2 learned v (not no-op). So Rule 4 never triggers; nobody re-suggests v.

Why none of the safety nets catch it

Rule 4 (re-suggest on no-op) fires only at the **proposer** whose value was overwritten by a no-op. The proposer is p2, and p2 learned v (not no-op). So Rule 4 never triggers; nobody re-suggests v .

Revocation's own Phase 1 would normally heal the split – a revoker collects ACKs, discovers v was accepted, and re-proposes it. But p1 wrote `learned[2] = no-op` through the `est_index` shortcut, and **every** slot-2 handler (including ACK collection) is gated on `learned =  $\perp$` .

Why none of the safety nets catch it

Rule 4 (re-suggest on no-op) fires only at the **proposer** whose value was overwritten by a no-op. The proposer is p2, and p2 learned v (not no-op). So Rule 4 never triggers; nobody re-suggests v .

Revocation's own Phase 1 would normally heal the split – a revoker collects ACKs, discovers v was accepted, and re-proposes it. But p1 wrote `learned[2] = no-op` through the `est_index` shortcut, and **every** slot-2 handler (including ACK collection) is gated on `learned =  $\perp$` .

The fabricated no-op freezes the slot before the safe path can ever run.

The inference treats an **absence** of a suggestion as a **no-op**.

Once a proposal can be **received and rejected** by a revoke, an absence is no longer trustworthy – so the optimization synthesizes a no-op over a slot the coordinator actually filled.

The fix

1. Safety – ballot guard in `fill_q_skips`.

Only learn a no-op for slot j when the local `prepared_ballot[j]` is still the owner's implicit $(0, q)$. If any higher ballot is present, a revoker is active and the gap-fill defers to consensus.

The fix

1. Safety – ballot guard in `fill_q_skips`.

Only learn a no-op for slot j when the local `prepared_ballot[j]` is still the owner's implicit $(0, q)$. If any higher ballot is present, a revoker is active and the gap-fill defers to consensus.

In Phase D, `prepared_ballot[2] = (1, p1) != (0, p2)` – so slot 2 is **not** learned as no-op.

The fix

1. Safety – ballot guard in `fill_q_skips`.

Only learn a no-op for slot j when the local `prepared_ballot[j]` is still the owner's implicit $(0, q)$. If any higher ballot is present, a revoker is active and the gap-fill defers to consensus.

In Phase D, `prepared_ballot[2] = (1, p1) != (0, p2)` – so slot 2 is **not** learned as no-op.

2. Liveness – NACK -> owner self-revoke.

A rejected ballot-0 SUGGEST returns a NACK carrying the higher ballot; the owner self-revokes, Phase 1 discovers the accepted value, and the slot converges.

The fix

1. Safety – ballot guard in `fill_q_skips`.

Only learn a no-op for slot j when the local `prepared_ballot[j]` is still the owner's implicit $(0, q)$. If any higher ballot is present, a revoker is active and the gap-fill defers to consensus.

In Phase D, `prepared_ballot[2] = (1, p1) != (0, p2)` – so slot 2 is **not** learned as no-op.

2. Liveness – NACK -> owner self-revoke.

A rejected ballot-0 SUGGEST returns a NACK carrying the higher ballot; the owner self-revokes, Phase 1 discovers the accepted value, and the slot converges.

Both together: opt 1+2+3 keeps Rule 2 throughput in the common case and falls back to proper Paxos when a revoke is active. See `Mencius_opt1_2_3_fixed.spur`.

Reproducing it

```
cargo run --release --manifest-path spur/Cargo.toml --bin spur -- \
  explore -e standard --config scheduler_configs/mencius_nocrash.json \
  -y --output-dir out bin/spur/mencius/Mencius_opt1_2_3.spur
./porcupine/main -input out -type duckdb -model kv -output-dir out
```

- Expected: **Some runs are NOT linearizable** – 8 of 708 runs, more than the ad-hoc opt1_2 variant because the β -window always revokes slot 5 too.
- `debug combined --run-id N` on a failing run shows `Revoke start: slot=2 ballot=(1,...)` and slot-2 divergence (no-op on the revoker vs committed uid elsewhere). The `_fixed` spec is linearizable.

Full write-up: `bug_opt1_2_3.md`.